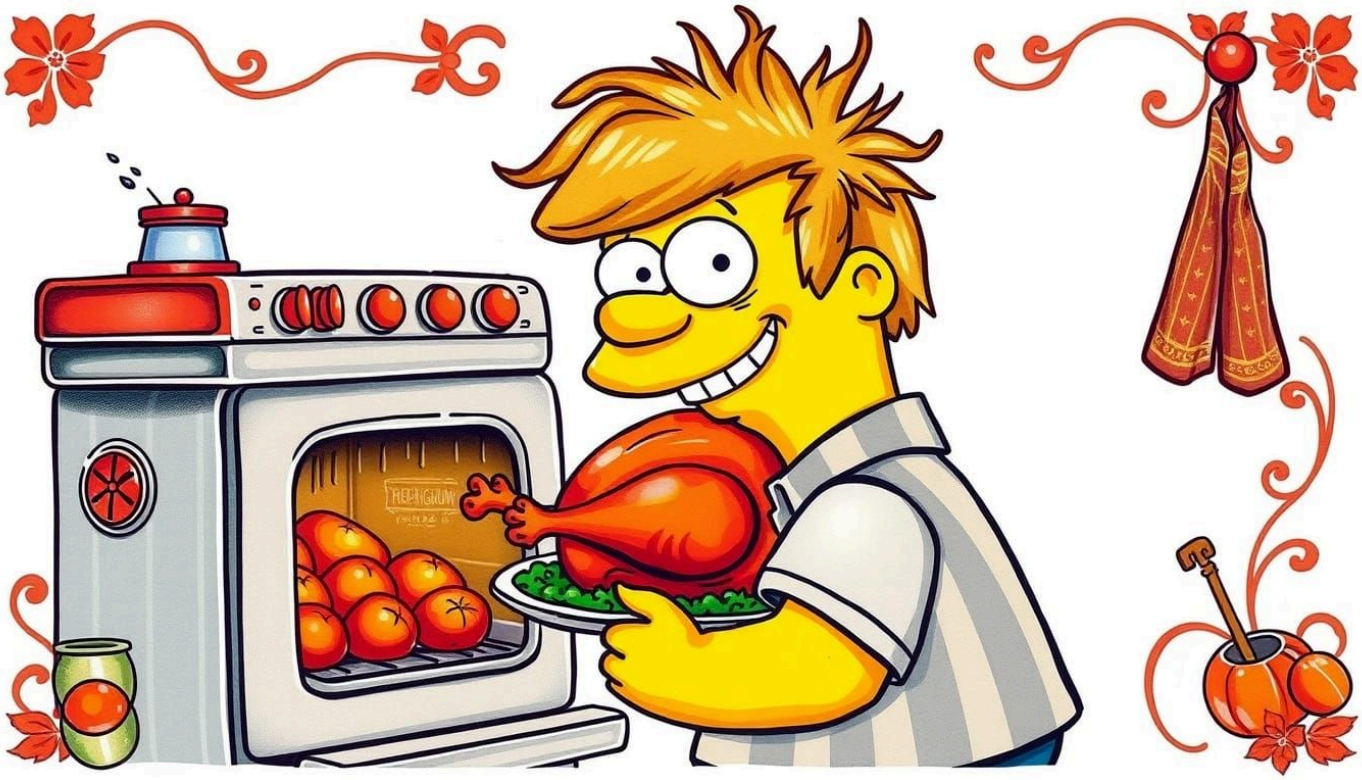
BY **GEOFFREY HUNTLEY** IN **AI** — 14 JUL 2025

Ralph Wiggum as a "software engineer"



> How Ralph Wiggum went from 'The Simpsons' to the biggest name in AI right now - Venture Beat

The Ralph Wiggum Loop from 1st principles (by the creator of Ralp
Geoffrey Huntley


Watch on

watch this to see it in practice and learn the theory; read below to go deeper

Ralph Wiggum (and why Claude Code's implementation isn't it) wit
Geoffrey Huntley


Watch on

watch this to learn why the claude code plugin isn't it 🤝

😎 Here's a cool little field report from a Y Combinator hackathon event where they put Ralph Wiggum to the test.

"We Put a Coding Agent in a While Loop and It Shipped 6 Repos Overnight"

https://github.com/repomirrorhq/repomirror/blob/main/repomirror.md

If you've seen my socials lately, you might have seen me talking about Ralph and wondering what Ralph is. Ralph is a technique. In its purest form, Ralph is a Bash loop.

```
while :; do cat PROMPT.md | claude-code ; done
```

Ralph can replace the majority of outsourcing at most companies for greenfield projects. It has defects, but these are identifiable and resolvable through various styles of prompts.

> **That's the beauty of Ralph - the technique is deterministically bad in an undeterministic world.**

Ralph can be done with any tool that does not cap tool calls and usage.

Ralph is currently building a brand new programming language. We are on the final leg before a brand new production-grade esoteric programming language is released. What's kind of wild to me is that Ralph has been able to build this language and is also able to program in this language without that language being in the LLM's training data set.

geoff
@GeoffreyHuntley · **Follow**

Amp creating a new programming language AFK

Watch on X

geoff
Amp creating a new programming language AFK

8:44 AM · Jul 13, 2025

Reply Copy link

Read more on X

Building software with Ralph requires a great deal of faith and a belief in eventual consistency. Ralph will test you. Every time Ralph has taken a wrong direction in making CURSED, I haven't blamed the tools; instead, I've looked inside. Each time Ralph does something bad, Ralph gets tuned - like a guitar.

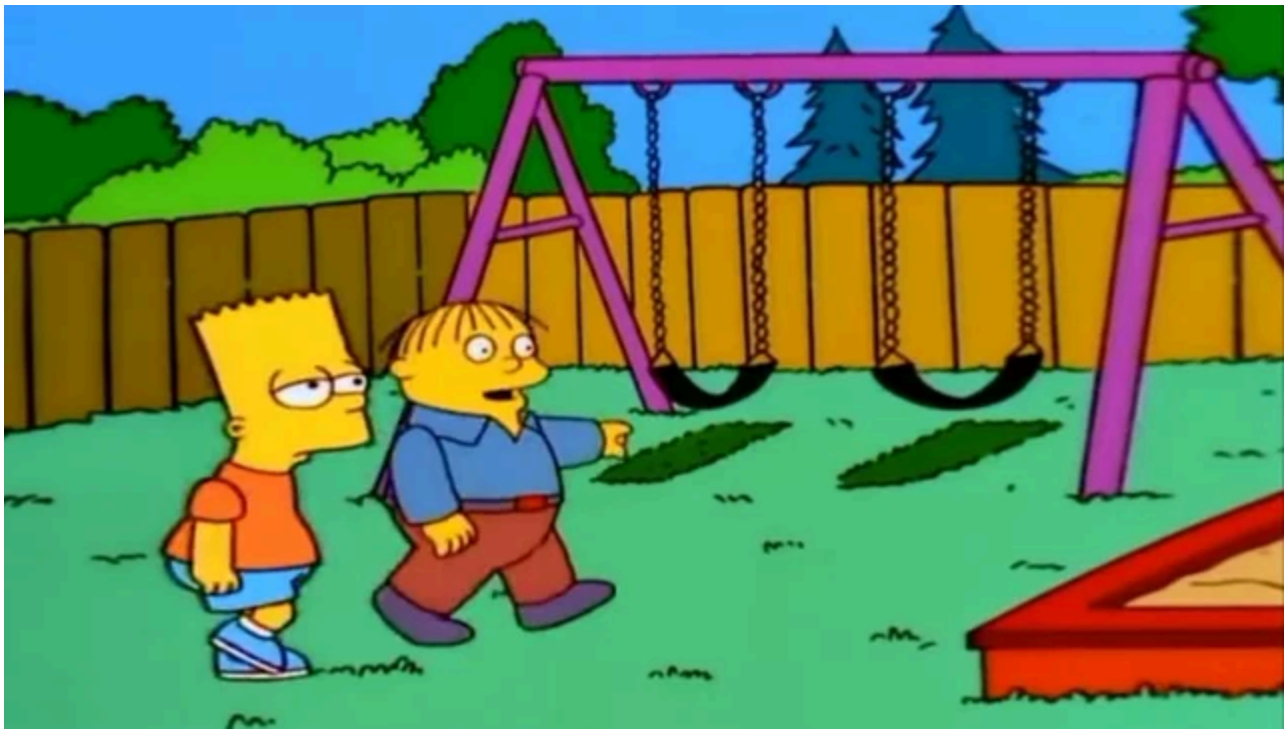
deliberate intentional practice

Something I've been wondering about for a really long time is, essentially, why do people say AI doesn't work for them? What do...

Geoffrey Huntley • Geoffrey Huntley

It begins with no playground, and Ralph is given instructions to construct one.

Ralph is very good at making playgrounds, but he comes home bruised because he fell off the slide, so one then tunes Ralph by adding a sign next to the slide saying “SLIDE DOWN, DON’T JUMP, LOOK AROUND,” and Ralph is more likely to look and see the sign.



Eventually all Ralph thinks about is the signs so that’s when you get a new Ralph that doesn't feel defective like Ralph, at all.

When I was in SFO, I taught a few smart people about Ralph. One incredibly talented engineer listened and used Ralph on their next contract, walking away with the wildest ROI. These days, all they think about is Ralph.

geoff
@GeoffreyHuntley · **Follow**

From my iMessage

(shared with permission)

Cost of a $50k USD contract, delivered, MVP, tested + reviewed with @ampcode.

$297 USD.

12:30 AM · Jul 11, 2025

176 Reply Copy link

Read 13 replies

what's in the prompt.md? can I have it?

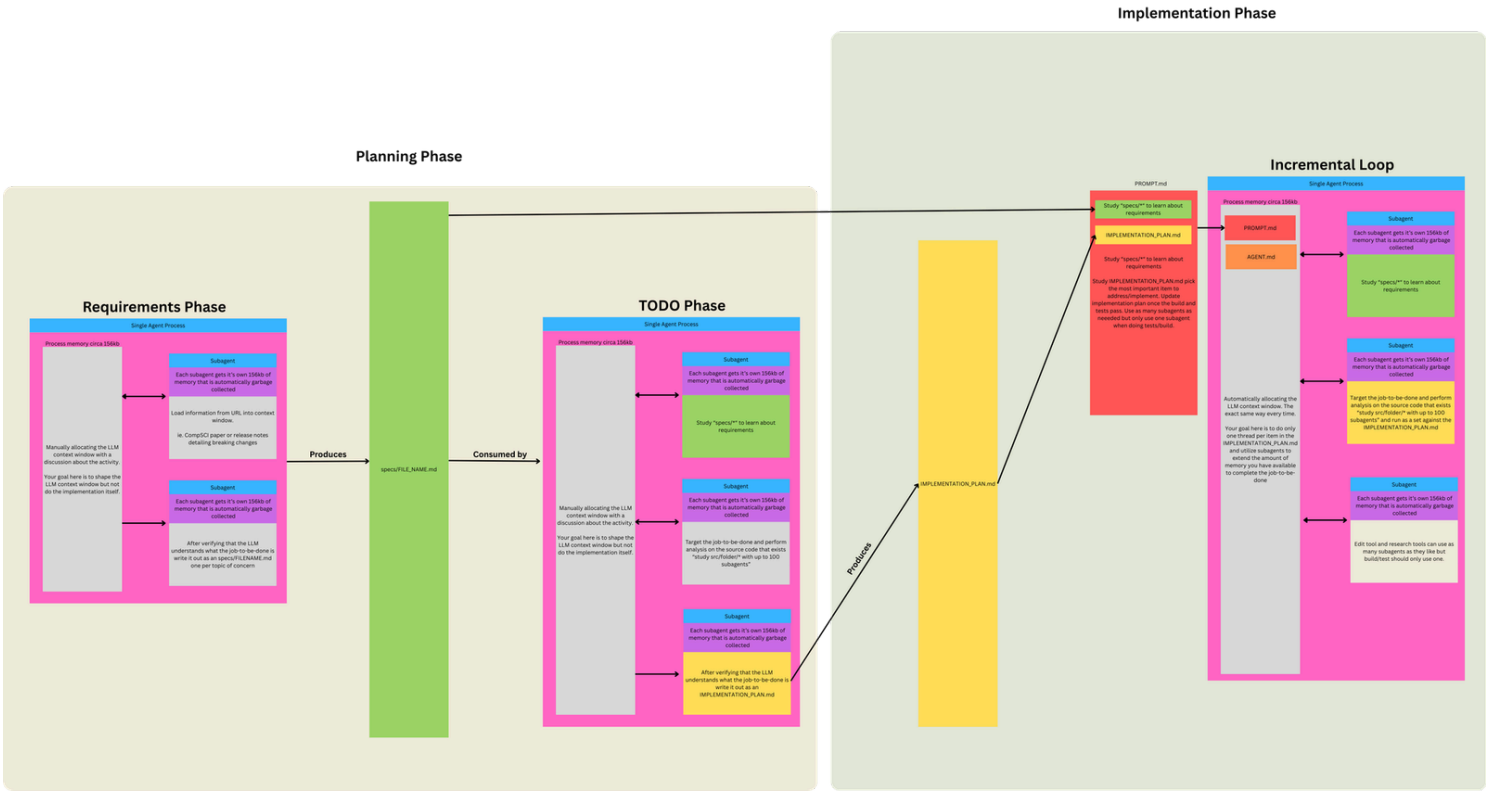
There seems to be an obsession in the programming community with the perfect prompt. There is no such thing as a perfect prompt.

Whilst it might be tempting to take the prompt from CURSED, it won't make sense unless you know how to wield it. You probably won't get the same outcomes by taking the prompt verbatim, because it has evolved through continual tuning based on observation of LLM behaviour. When CURSED is being built, I'm sitting there watching the stream, looking for patterns of bad behaviour—opportunities to tune Ralph.

first some fundamentals

While I was in SFO, everyone seemed to be trying to crack on multi-agent, agent-to-agent communication and multiplexing. At this stage, it's not needed. Consider microservices and all the complexities that come with them. Now, consider what microservices would look like if the microservices (agents) themselves are non-deterministic—a red hot mess.

What's the opposite of microservices? A monolithic application. A single operating system process that scales vertically. Ralph is monolithic. Ralph works autonomously in a single repository as a single process that performs one task per loop.



the ralph wiggum technique as a diagram

To get good outcomes with Ralph, you need to ask Ralph to do one thing per loop. **Only one thing**. Now, this might seem wild, but you also need to trust Ralph to decide what's the most important thing to implement. This is full hands-off vibe coding that will test the bounds of what you consider "responsible engineering".

LLMs are surprisingly good at reasoning about what is important to implement and what the next steps are.

> Your task is to implement missing stdlib (see @specs/stdlib/*) and compiler functionality and produce an compiled application in the cursed language via LLVM for that functionality using parrallel subagents. Follow the @fix_plan.md and **choose the most important thing**.

There's a few things in the above prompt which I'll expand upon shortly but the other key thing is **deterministically allocate the stack the same way every loop**.

0a. study specs/* to learn about the compiler specifications

0b. The source code of the compiler is in src/

0c. study fix_plan.md.

1. Your task is to implement missing stdlib (see @specs/stdlib/*) and compiler functionality and produce an compiled application in the cursed language via LLVM for that functionality using parrallel subagents. Follow the fix_plan.md and choose the most important 10 things. Before making changes search codebase (don't assume not implemented) using subagents. You may use up to 500 parrallel subagents for all operations but only 1 subagent for build/tests of rust.

The items that you want to allocate to the stack every loop are your plan ("@fix_plan.md") and your specifications. See below if specs are a new concept to you.

From Design doc to code: the Groundhog AI coding assistant (and new Cursor vibecoding meta)

Ello everyone, in the “Yes, Claude Code can decompile itself. Here’s the source code” blog post, I teased about a new meta when usin...

Geoffrey Huntley • Geoffrey Huntley

Specs are formed through a conversation with the agent at the beginning phase of a project. Instead of asking the agent to implement the project, what you want to do is have a long conversation with the LLM about your requirements for what you're about to implement. Once your agent has a decent understanding of the task to be done, it's at that point that you issue a prompt to write the specifications out, one per file, in the specifications folder.

one item per loop

One item per loop. I need to repeat myself here—one item per loop. You may relax this restriction as the project progresses, but if it starts going off the rails, then you need to narrow it down to just one item.

The name of the game is that you only have approximately 170k of context window to work with. So it's essential to use as little of it as possible. The more you use the context window, the worse the outcomes you'll get. Yes, this is wasteful because you're effectively burning the allocation of the specifications every loop and not reusing the allocation.

extend the context window

The way that agentic loops work is by executing a tool and then evaluating the result of that tool. The evaluation results in an allocation being added to your context window. See below.

autoregressive queens of failure

Have you ever had your AI coding assistant suggest something so off-base that you wonder if it's trolling you? Welcome to the world...

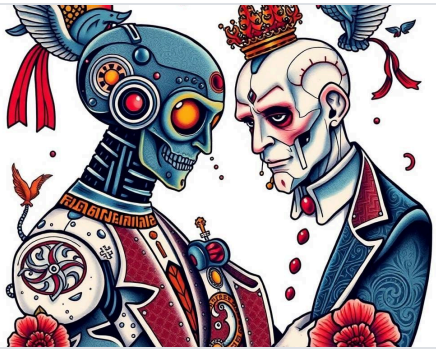
Geoffrey Huntley • Geoffrey Huntley

Ralph requires a mindset of not allocating to the primary context window. Instead, what you should do is spawn subagents. Your primary context window should operate as a scheduler, scheduling other subagents to perform expensive allocation-type work, such as summarising whether your test suite worked.

I dream about AI subagents; they whisper to me while I'm asleep

In a previous post, I shared about "real context window" sizes and "advertised context window sizes" Claude 3.7's advertised context...

Geoffrey Huntley • Geoffrey Huntley



> Your task is to implement missing stdlib (see @specs/stdlib/*) and compiler functionality and **produce an compiled application in the cursed language via LLVM for that functionality using parrallel subagents**. Follow the fix_plan.md and choose the most important thing. Before making changes search codebase (don't assume not implemented) using subagents. **You may use up to parrallel subagents for all operations but only 1 subagent for build/tests of rust.**

Another thing to realise is that you can control the amount of parallelism for subagents.

0:00 / 0:20 1×

84 squee (claude subagents) chasing <T>

If you were to fan out to a couple of hundred subagents and then tell those subagents to run the build and test of an application, what you'll get is bad form back pressure. Thus, the instruction above is that only a single subagent should be used for validation, but Ralph can use as many subagents as he likes for searching the file system and for writing files.

don't assume it's not implemented

The way that all these coding agents work is via `ripgrep`, and it's essential to understand that code-based search can be non-deterministic.

from Luddites to AI: the Overton Window of disruption

I've been thinking about Overton Windows lately, but not of the political variety. You see, the Overton window can be adapted to...

Geoffrey Huntley • Geoffrey Huntley

A common failure scenario for Ralph is when the LLM runs `ripgrep` and comes to the incorrect conclusion that the code has not been implemented. This failure scenario is easily resolved by erecting a sign for Ralph, instructing Ralph not to make assumptions.

> Before making changes search codebase (don't assume an item is not implemented) using parrallel subagents. Think hard.

If you wake up to find that Ralph is doing multiple implementations, then you need to tune this step. This nondeterminism is the Achilles' heel of Ralph.

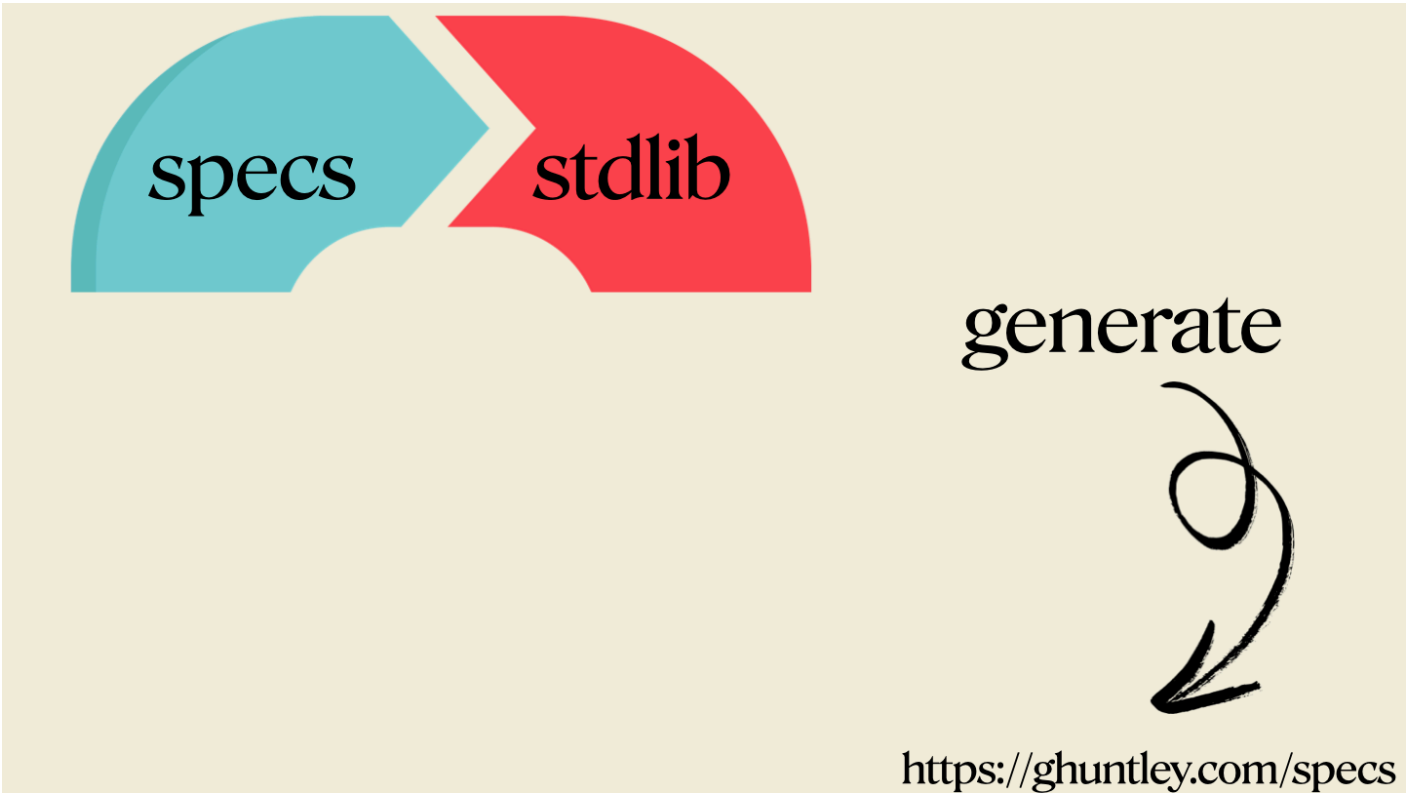
phase one: generate

Generating code is now cheap, and the code that Ralph generates is within your complete control through your technical standard library and your specifications.

From Design doc to code: the Groundhog AI coding assistant (and new Cursor vibecoding meta)

Ello everyone, in the “Yes, Claude Code can decompile itself. Here’s the source code” blog post, I teased about a new meta when usin...

Geoffrey Huntley • Geoffrey Huntley



You are using Cursor AI incorrectly...

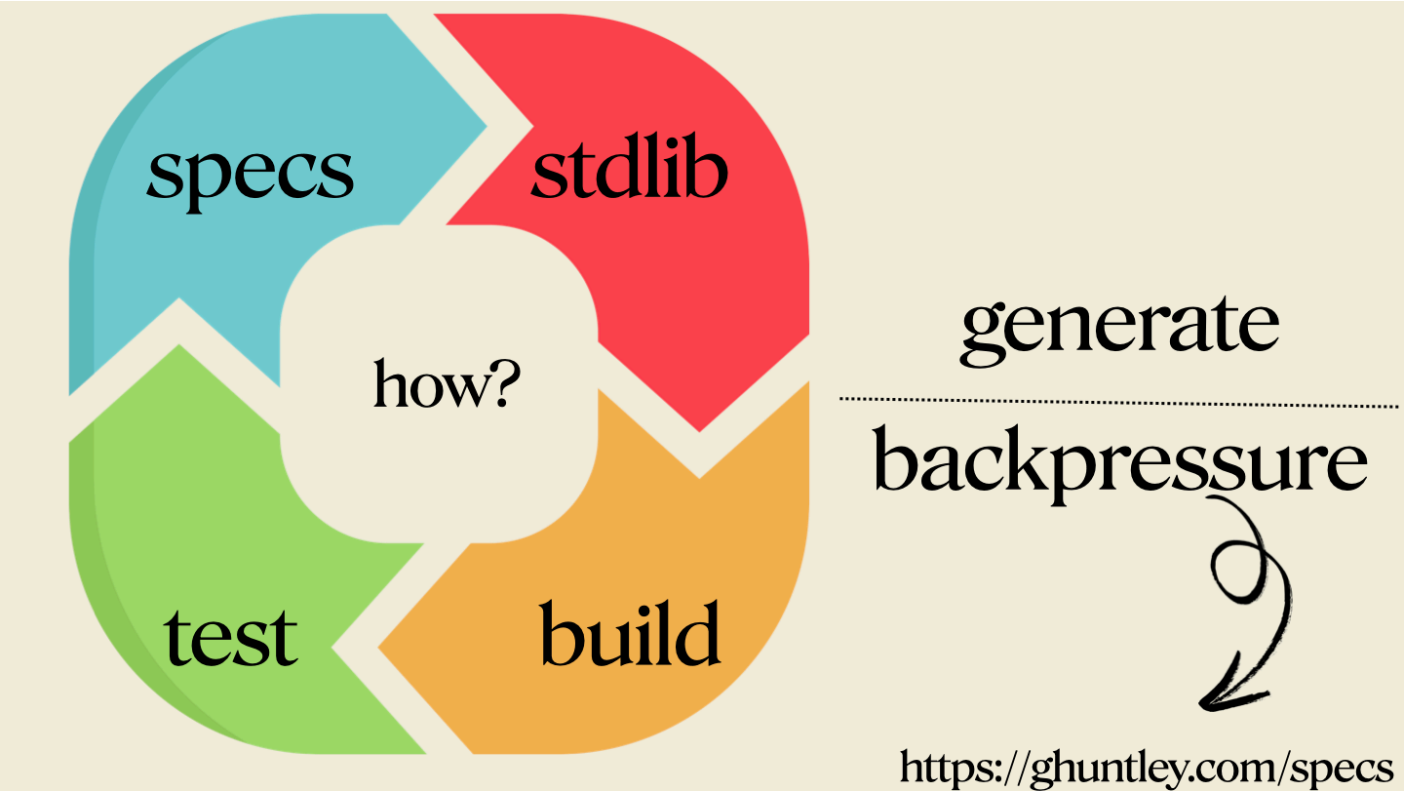
🩹 I recently shipped a follow-up blog post to this one; this post remains true. You’ll need to know this to be able to drive the N-...

Geoffrey Huntley • Geoffrey Huntley

If Ralph is generating the wrong code or using the wrong technical patterns, then you should update your standard library to steer it to use the correct patterns.

If Ralph is building the wrong thing completely, then your specifications may be incorrect. A big, hard lesson for me when building CURSED was that it was only a month in that I noticed that my specification for the lexer defined a keyword twice for two opposing scenarios, which resulted in a lot of time wasted. Ralph was doing stupid shit, and I guess it's easy to blame the tools instead of the operator.

phase two: backpressure



This is where you need to have your engineering hat on. As code generation is easy now, what is hard is ensuring that Ralph has generated the right thing. Specific programming languages have inbuilt back pressure through their type system.

Now you might be thinking, "Rust! It's got the best type system." However, one thing with Rust is that the compilation speed is slow. It's the speed of the wheel turning that matters, balanced against the axis of correctness.

Which language to use requires experimentation. As I'm creating a compiler, I wanted extreme correctness, which meant using Rust; however, this approach means it's built more slowly. These LLMs are not very good at generating the perfect Rust code in one attempt, which means they need to make more attempts.

That can be either a positive or a negative thing.

In the diagram above, it just shows the words "test and build", but this is where you put your engineering hat on. Anything can be wired in as back pressure to reject invalid code generation. That could be security scanners, it could be static analysers, it could be anything. But the key collective sum is that the wheel has got to turn fast.

A staple when building CURSED has been the following prompt. After making a change, run a test just for that unit of code that was implemented and improved.

> **After implementing functionality or resolving problems, run the tests for that unit of code that was improved**.

If you're using a dynamically typed language, I must stress the importance of wiring in a static analyser/type checker when Ralphing, such as:

- https://www.erlang.org/doc/apps/dialyzer/dialyzer.html
- https://pyrefly.org/

If you do not, then you will run into a bonfire of outcomes.

capture the importance of tests in the moment

When you instruct Ralph to write tests as a form of back pressure, because we are writing Ralph doing one thing and one thing only, every loop, with each loop with its new context window, it's crucial in that moment to ask Ralph to write out the meaning and the importance of the test explaining what it's trying to do.

> Important: When authoring documentation (ie. rust doc or cursed stdlib documentation) capture the why tests and the backing implementation is important.

In implementation, it looks similar to this. To me, I see it like leaving little notes for future iterations by the LLM, explaining why a test exists and its importance because future loops will not have the reasoning in their context window.

```
defmodule Anole.Database.QueryOptimizerTest do
  @moduledoc """
  Tests for the database query optimizer.

  These tests verify the functionality of the QueryOptimizer module, ensuri
  it correctly implements caching, batching, and analysis of database queri
  improve performance.

  The tests use both real database calls and mocks to ensure comprehensive
  while maintaining test isolation and reliability.
  """

  use Anole.DataCase

  import ExUnit.CaptureLog
  import Ecto.Query
  import Mock

  alias Anole.Database.QueryOptimizer
  alias Anole.Repo
  alias Anole.Tenant.Isolator
  alias Anole.Test.Factory

  # Set up the test environment with a tenant context
  setup do
    # Create a tenant for isolation testing
    tenant = Factory.insert(:tenant)
```

```
    # Ensure the optimizer is initialized
    QueryOptimizer.init()

    # Return context
    {:ok, %{tenant: tenant}}
  end

  describe "init/0" do
    @doc """
    Tests that the QueryOptimizer initializes the required ETS tables.

    This test ensures that the init function properly creates the ETS table
    needed for caching and statistics tracking. This is fundamental to the
    module's operation.
    """
    test "creates required ETS tables" do
      # Clean up any existing tables first
      try do :ets.delete(:anole_query_cache) catch _:_ -> :ok end
      try do :ets.delete(:anole_query_stats) catch _:_ -> :ok end

      # Call init
      assert :ok = QueryOptimizer.init()

      # Verify tables exist
      assert :ets.info(:anole_query_cache) != :undefined
      assert :ets.info(:anole_query_stats) != :undefined

      # Verify table properties
      assert :ets.info(:anole_query_cache, :type) == :set
      assert :ets.info(:anole_query_stats, :type) == :set
    end
  end
```

I've found that it helps the LLMs decide if a test is no longer relevant or if the test is important, and it affects the decision-making whether to delete, modify or resolve a test [failure].

no cheating

Claude has the inherent bias to do minimal and placeholder implementations. So, at various stages in the development of CURSED, I've brought in a variation of this prompt.

> After implementing functionality or resolving problems, run the tests for that unit of code that was improved. **If functionality is missing then it's your job to add it as per the application specifications**. Think hard.
>
> If tests unrelated to your work fail then it's your job to resolve these tests as part of the increment of change.
>
> 999999999999999999999999999. DO NOT IMPLEMENT PLACEHOLDER OR SIMPLE IMPLEMENTATIONS. WE WANT FULL IMPLEMENTATIONS. **DO IT OR I WILL YELL AT YOU**

Do not be dismayed if, in the early days, Ralph ignores this sign and does placeholder implementations. The models have been trained to chase their reward function, and the reward function is compiling code. You can always run more Ralphs to identify placeholders and minimal implementations and transform that into a to-do list for future Ralph loops.

the todo list

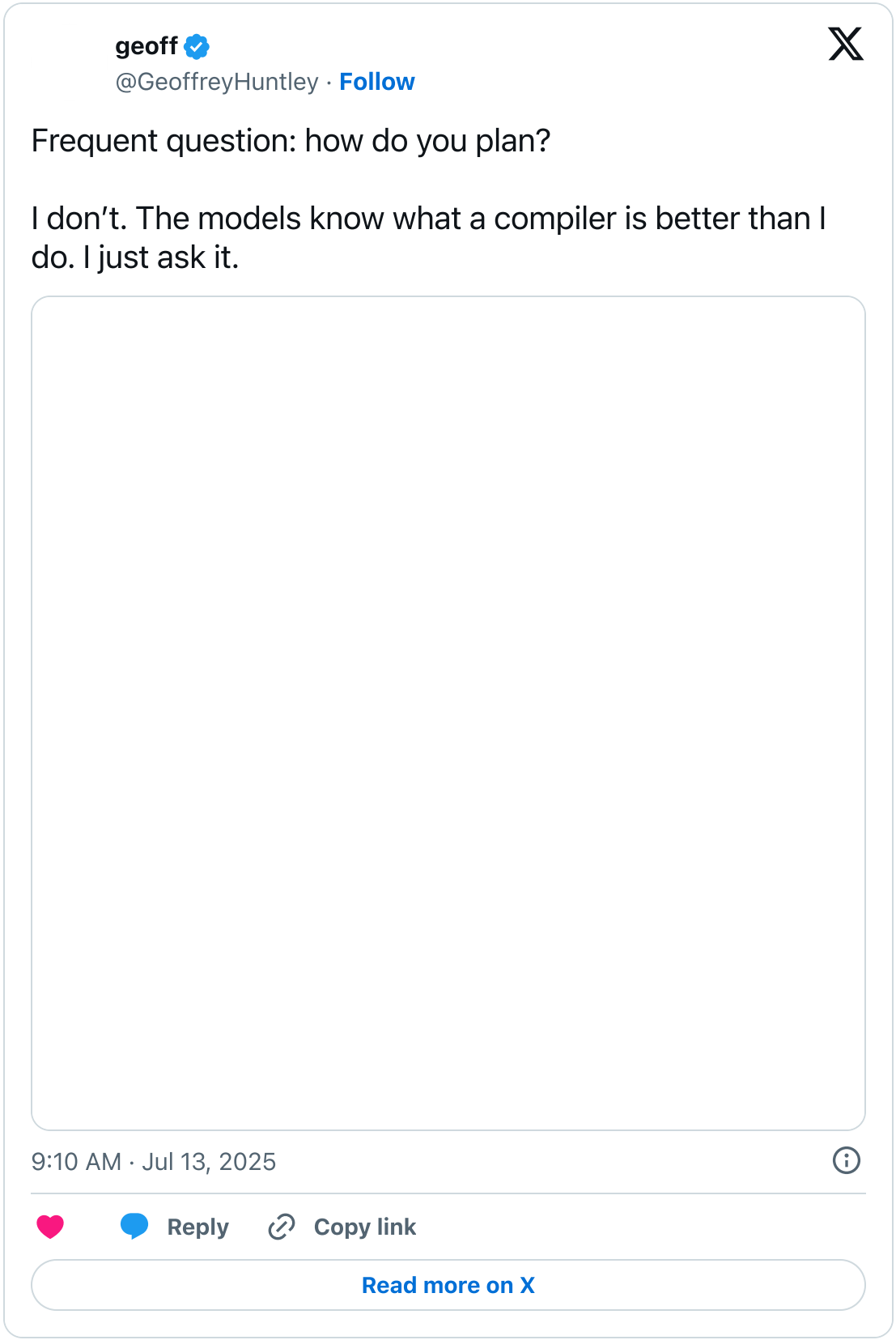
Speaking of which, here is the prompt stack I've been using over the last couple of weeks to build the TODO list. This is the part where I say Ralph will test you. You have to believe in eventual consistency and know that most issues can be resolved through more loops with Ralph, focusing on the areas where Ralph is making mistakes.

> study specs/* to learn about the compiler specifications and fix_plan.md to understand plan so far.
>
> The source code of the compiler is in src/*

> The source code of the examples is in examples/* and the source code of the tree-sitter is in tree-sitter/*. Study them.
>
> The source code of the stdlib is in src/stdlib/*. Study them.
>
> First task is to study @fix_plan.md (it may be incorrect) and is to use up to 500 subagents to study existing source code in src/ and compare it against the compiler specifications. From that create/update a @fix_plan.md which is a bullet point list sorted in priority of the items which have yet to be implemeneted. Think extra hard and use the oracle to plan. **Consider searching for TODO, minimal implementations and placeholders**. Study @fix_plan.md to determine starting point for research and keep it up to date with items considered complete/incomplete using subagents.
>
> Second task is to use up to 500 subagents to study existing source code in examples/ then compare it against the compiler specifications. From that create/update a fix_plan.md which is a bullet point list sorted in priority of the items which have yet to be implemeneted. Think extra hard and use the oracle to plan. Consider searching for TODO, minimal implementations and placeholders. Study fix_plan.md to determine starting point for research and keep it up to date with items considered complete/incomplete.
>
> IMPORTANT: The standard library in src/stdlib should be built in cursed itself, not rust. If you find stdlib authored in rust then it must be noted that it needs to be migrated.
>
> ULTIMATE GOAL we want to achieve a self-hosting compiler release with full standard library (stdlib). Consider missing stdlib modules and plan. If the stdlib is missing then author the specification at specs/stdlib/FILENAME.md (do NOT assume that it does not exist, search before creating). The naming of the module should be GenZ named and not conflict with another stdlib module name. If you create a new stdlib module then document the plan to implement in @fix_plan.md

Eventually, Ralph will run out of things to do in the TODO list. Or, it goes completely off track. It's Ralph Wiggum, after all. It's at this stage where it's a matter of taste. Through building of CURSED, I have deleted the TODO list multiple times. The TODO list is what I'm watching like a hawk. And I throw it out often.

Now, if I throw the TODO list out, you might be asking, "Well, how does it know what the next step is?" Well, it's simple. You run a Ralph loop with explicit instructions such as above to generate a new TODO list.



Then when you've got your todo list you kick Ralph back off again with... instructions to switch from planning mode to building mode...

loop back is everything

You want to program in ways where Ralph can loop himself back into the LLM for evaluation. This is incredibly important. Always look for opportunities to loop Ralph back on itself. This could be as simple as instructing it to add additional logging, or in the case of a compiler, asking Ralph to compile the application and then looking at the LLVM IR representation.

> You may add extra logging if required to be able to debug the issues.

ralph can take himself to university

The @AGENT.md is the heart of the loop. It instructs how Ralph should compile and run the project. If Ralph discovers a learning, permit him to self-improve:

> When you learn something new about how to run the compiler or examples make sure you update @AGENT.md using a subagent but keep it brief. For example if you run commands multiple times before learning the correct command then that file should be updated.

During a loop, Ralph might determine that something needs to be fixed. It's crucial to capture that reasoning.

> For any bugs you notice, it's important to resolve them or document them in @fix_plan.md to be resolved using a subagent even if it is unrelated to the current piece of work after documenting it in @fix_plan.md

you will wake up to a broken code base

Yep, it's true, you'll wake up to a broken codebase that doesn't compile from time to time, and you'll have situations where Ralph can't fix it himself. This is where you need to put your brain on. You need to make a judgment call. Is it easier to do a `git reset --`

`hard` and to kick Ralph back off again? Or do you need to come up with another series of prompts to be able to rescue Ralph?

> When the tests pass update the @fix_plan.md`, then add changed code and @fix_plan.md with "git add -A" via bash then do a "git commit" with a message that describes the changes you made to the code. After the commit do a "git push" to push the changes to the remote repository.
>
> As soon as there are no build or test errors create a git tag. If there are no git tags start at 0.0.0 and increment patch by 1 for example 0.0.1 if 0.0.0 does not exist.

I recall when I was first getting this compiler up and running, and the number of compilation errors was so large that it filled Claude's context window. So, at that point, I took the file of compilation errors and threw it into Gemini, asking Gemini to create a plan for Ralph.

but maintainability?

When I hear that argument, I question “by whom”? By humans? Why are humans the frame for maintainability? Aren’t we in the post-AI phase where you can just run loops to resolve/adapt when needed? 😎

any problem created by AI can be resolved through a different series of prompts

Which brings me to this point. If you wanted to be cheeky, you could probably find the codebase for CURSED on GitHub. I ask that you refrain from sharing it on social media, as it's not yet ready for launch. I want to dial this thing in so much that we have indisputable proof that AI can build a brand new programming language and program a programming language where it has no training data in its training set is possible.

cursed as a webserver

What I'd like people to understand is that all these issues, created by Ralph, can be resolved by crafting a different series of prompts and running more loops with Ralph.

I'm expecting CURSED to have some significant gaps, just like Ralph Wiggum. It'd be so easy for people to poke holes in CURSED, as it is right now, which is why I have been holding off on publishing this post. The repository is full of garbage, temporary files, and binaries.

> Ralph has three states. Under baked, baked, or baked with unspecified latent behaviours (which are sometimes quite nice!)

When CURSED ships, understand that Ralph built it. What comes next, technique-wise, won't be Ralph. I firmly maintain that if models and tools remain as they are now, we are in post-AGI territory. All you need are tokens; these models yearn for tokens, so throw them at them, and you have primitives to automate software development if you take the right approaches.

Having said all of that, engineers are still needed. There is no way this is possible without senior expertise guiding Ralph. Anyone claiming that engineers are no longer required and a tool can do 100% of the work without an engineer is peddling horseshit.

However, the Ralph technique is surprisingly effective enough to displace a large majority of SWEs as they are currently for Greenfield projects.

As a final closing remark, I'll say,

> "There's no way in heck would I use Ralph in an existing code base"

though, if you try, I'd be interested in hearing what your outcomes are. This works best as a technique for bootstrapping Greenfield, with the expectation you'll get 90% done with it.

current prompt used to build cursed

Here's the current prompt used by Ralph to build CURSED.

```
0a. study specs/* to learn about the compiler specifications

0b. The source code of the compiler is in src/

0c. study fix_plan.md.

1. Your task is to implement missing stdlib (see @specs/stdlib/*) and compi
```

```
2. After implementing functionality or resolving problems, run the tests fo

2. When you discover a parser, lexer, control flow or LLVM issue. Immediate

3. When the tests pass update the @fix_plan.md`, then add changed code and

999. Important: When authoring documentation (ie. rust doc or cursed stdlib

9999. Important: We want single sources of truth, no migrations/adapters. I

999999. As soon as there are no build or test errors create a git tag. If t

999999999. You may add extra logging if required to be able to debug the is

9999999999. ALWAYS KEEP @fix_plan.md up to do date with your learnings usin

99999999999. When you learn something new about how to run the compiler or

999999999999. IMPORTANT DO NOT IGNORE: The standard libray should be author

99999999999999. IMPORTANT when you discover a bug resolve it using subagent

9999999999999999. When you start implementing the standard library (stdlib)

99999999999999999. The tests for the cursed standard library "stdlib" shoul

999999999999999999. Keep AGENT.md up to date with information on how to bu

9999999999999999999. For any bugs you notice, it's important to resolve t

99999999999999999999. When authoring the standard library in the cursed
```

```
99999999999999999999999999. When @fix_plan.md becomes large periodically cl

99999999999999999999999999. If you find inconsistentcies in the specs/* the

999999999999999999999999999999. DO NOT IMPLEMENT PLACEHOLDER OR SIMPLE IMPLEM

9999999999999999999999999999999999. SUPER IMPORTANT DO NOT IGNORE. DO NOT PLAC
```

current prompt used to plan cursed

```
study specs/* to learn about the compiler specifications and fix_plan.md to

The source code of the compiler is in src/*

The source code of the examples is in examples/* and the source code of the

The source code of the stdlib is in src/stdlib/*. Study them.

First task is to study @fix_plan.md (it may be incorrect) and is to use up

Second task is to use up to 500 subagents to study existing source code in

IMPORTANT: The standard library in src/stdlib should be built in cursed its

ULTIMATE GOAL we want to achieve a self-hosting compiler release with full
```

ps. socials

- X - https://x.com/GeoffreyHuntley/status/1944614322107564194
- LinkedIn - https://www.linkedin.com/posts/geoffreyhuntley_ralph-wiggum-as-a-software-engineer-activity-7350383201233608705-vBRf
- Bsky - https://bsky.app/profile/ghuntley.com/post/3ltvkz6gkh22g

p.p.s don't miss out on free lifetime access

Join thousands getting premium insights delivered weekly. **Subscribe now and you'll never pay** - even when I start charging new subscribers. This free-for-life offer won't last forever.

Subscribe to Newsletter

PREVIOUS

Claude Sonnet is a small-brained mechanical squirrel of <T>

NEXT

source code analysis of Amazon Kiro

YOU MIGHT ALSO LIKE...

AI

cognitive security

This is a follow-up from my previous post about AI as an

READ MORE

AI

AI as economic warfare

People often ask me how I feel about open-source models, and to

READ MORE

AI

porting software has been trivial for a while now. here's how you do it.

This one is short and sweet. if you want to port a

READ MORE

AI

A couple of days ago, I sat down with Vivek Bharathi and dumped my brains. Here's the interview...

0:00 /11:14 1× Below you'll find an AI

READ MORE